ASML	**Modified date:** 2025-11-27 **Status:** UCC **Info type:** EPS	**Document name:** D001047842/09

Program/Machine type:	**Project/Group:** mpc

Title:

EPS Context Model

Summary:

This document describes:

- the core Context Model for wafer fab process
- different levels of the Context Model to keep the visibility clear for high level and detailed overview
- definition of core model entities and their contexts
- the relationships among different process entities along with cardinality
- how a process entity can be uniquely identified in the platform

Export control status: Company

For change control		Reviewers		For info	
Achref Kilani Jrad	FCO-205	Rinie Egas	FC-226	Helmar Behrends	PO D2
		Arch		Valentin Boss	PO D3
		Jaap Karssenberg	LIS SE	Pelagia Sykoudi	PO Orpheus
		Arch			
		Abelneh Teka	Epic Arch		
		Frans van der Meeren	SUB-FCA-205		
		Justin Verkuijl	SUB-FCA-208		
		Tim Nitzsche	D Layer Arch		

Maintainer	**: Kamran Ahmed**	**Template Name:**
Pages incl. cover	**: 44**	Component-EPS **Rev.:** 016
Word processor	**: MS Word 2010**	**Owner:** Henk Kant

© ASML 2021. All rights reserved.
Reproduction in whole or in part is prohibited without the prior written consent of ASML.

1

CONTRIBUTING AUTHORS

Name author	Part(s) of this document
Frans van der Meeren	Initial baseline
Trosky Boris Callo Arias	Baseline for PI15
Trosky Boris Callo Arias	Context Model and inventory sync, overlay and alignment requirements
Kamran Ahmed	Extension-related updates and modification
Kamran Ahmed	Representation of Context entity identifier
Kamran Ahmed	New relational interface for Context Model in Analytics Database.

2

OPEN ISSUES

ID	Description	IP/ECR AIR-ID	Action holder	Resolution Date
OI02	Design algorithm for shortest unambiguous path through the context model graph from an anchor point to an entity-attribute leaf, needed for extensibility. **In the scope of:** SIA Context Model extension mechanism-D001179131		Kamran Ahmed	
OI05	**Execution Flow context attributes** First version addressed by v1.1.0 - Common TDBB context flow computejob_flowexecution_commontdbb_tdbblevels = the given setting computejob_flowexecution_commontdbb_tdbblevel = the row in the trellis plot so which level computejob_flowexecution_commontdbb_virtualdatasetid = the values denoting a cell in the trellis (e.g. "<Context_Model_path_lotId>=123, <Context_Model_path_chuckId>=1" etc) - Any Flow can create/calculate new context, i.e. different from its settings. - Flow settings in Inventory or Dataset? (Rinie) - Rename Name -> flow type name ? (Patrick)		Frans van der Meeren/Kamran Ahmed	
OI06	Extended uses cases for ComputeJobs that need to be supported, either by explicit attributes and relationships in the context model or by queries in the Inventory		Kamran Ahmed	
OI07	Die association with intrafield points intraDiePosition to enable slicing without dedicated business logic in the frontend to recalculate wafer level coordinates		Kamran Ahmed	
OI09	Classification/type of computeJobs, i.e. LISJobs Feedback by HVM architects points that LISJobs is not the most suitable, hence a dedicated alignment is needed.		Kamran Ahmed	
OI10	Next baseline will be that all datasetreferences have Flow as anchor-point. Path addressing will also be revised		Kamran Ahmed	
OI13	Modeling Virtual wafers. A virtual (aggregated) wafer can also be derived from one or more compute jobs. We need to support cascading of multiple processing functions (jobs)		Kamran Ahmed	

© ASML 2021 Company Secret

ID	Description	IP/ECR AIR-ID	Action holder	Resolution Date
OI14	OTASJob addition is temporary in core model because there is no explicit platform support except some functionality in DPF to access documents from OTAS DAMA. It might be removed in future. Platform needs to provide more freedom to plugins to extend the core context model. See: [LISA-40295] [Design] Data plugin triggers for data (docs) available independent of ADELler/metro - Jira (asml.com))		Kamran Ahmed	
OI15	Using current Context Model and description, only attribute with prefix 'Key' is considered as unique criteria for an Entity which is not always correct. E.g. in 'Document' the attribute 'KeyDocumentId' is not the only unique criteria as OBST can accept another type of document with same KeyDocumentId (canonical key). Similarly LISJob can only be uniquely identified via composite key but there is no mentioning of those composite attributes in EPS. Moreover, we also need to address the design approach of how a composite key is to be defined e.g. as a separate class or by stereotyping. Stereotyping can be confusing in cases of unique attributes driven from parent and participating in composite key. Usage of word 'Key' is to be addressed with other cases such as KeyLotId, KeyLayerId and KeyWaferId which are being used differently in Analytics.		Kamran Ahmed	24Q2
OI16	Introduction of new enum MeasurementType which has been reverted back from v7 is to be resolved with domain expert if it was really a subtype of MeasureProcessJob or part of another ProcessJob in Fab.		Kamran Ahmed	24Q3

1

2 # DOCUMENT CHANGE & HISTORY RECORD

Maintainer	Last update	Status	Version	SPS/IP/ECR, AIR-ID	Comment
Frans van der Meeren	2021-06-24	Draft	0.1		Creation
Frans van der Meeren	2021-07-29	Draft	0.1		For Review
Trosky Boris Callo Arias	2021-08-05	Review	0.1		Baseline for PI15
Trosky Boris Callo Arias	2021-11-22	Review	0.1		UCC
Trosky Boris Callo Arias	2021-11-23	Draft	1.0		Overlay support, consistency with inventory, and sub-wafer level context
Trosky Boris Callo Arias	2022-04-11	Draft	1.0		Include review feedback for overlay support, and sub-wafer level context
Trosky Boris Callo Arias	2022-05-02	Review	1.0		Include review feedback from review in wk22.16 For UCC and Semantic version (major.minor.patch), so **1.0.0**
Trosky Boris Callo Arias	2022-05-13	UCC	1.0		UCC
Trosky Boris Callo Arias	2022-06-21	Draft	1.1.0		Scope for 22Q3
Trosky Boris Callo Arias	2022-06-21	For Review	1.1.0		Scope for 22Q3: - LIS computeJob minor updates

Maintainer	Last update	Status	Version	SPS/IP/ECR, AIR-ID	Comment
					- ~~Flow concept~~ - Minor Inventory/ Context Model syncs: measures and latestExposure - Naming of attributes : This consider as major change, hence a new version is required Context Model 2.0.0
Trosky Boris Callo Arias	2022-10-20	For Review	2.0.0		Rollback Flow context design to enable FLOW CRUD (containment) Die layout for cEPE (containment). SIMULATION strategy type for ALO12C
Kamran Ahmed	2023-11-30	For Review	2.1.0		OTASJob extension from ComputeJob - Addition of ControlStrategyType Enum - Addition of ControlFunctionType Enum Visibility corrections to InputJobs relation
Kamran Ahmed	2024-01-23	UCC	2.1.0		UCC
Kamran Ahmed	2024-02-08	For Review	2.1.1		- Added guideline for Enum usages and declaration in Context Model along with ChuckIdType value modification. - Small correction in stakeholders list - Added some OI to be resolved next quarters - Created a dependency between ExposureLogicalWafer and WaferPoint on request of IMO3 and allowing other domain also use that relation. - Updated MeasuredLogicalWafer to WaferPoint cardinality from 1..6 (which was ALO specific) to more generic cardinality of 1..n to be used in other domain too.
Kamran Ahmed	2024-02-29	UCC	2.1.1		UCC
Kamran Ahmed	2024-05-17	UCC	2.1.2		- Updated context model to represent Context entity identifiers.
Kamran Ahmed	2025-10-27	For Review	2.1.3		- Lot Rework related fields added in ExposureProcessJob and WaferExposureProcessJob - Entity Relationship Diagram added for entities from Analytics Database, those mapped to Core Context. - One to one mapping from Core Context Model contexts to individual fields in Relational tables in Analytics Database. - Flow has been removed as it was not being supported yet by platform.
Kamran Ahmed	2025-11-27	UCC	2.1.3		-

APPLICABLE DOCUMENTS

Controlling documents

Ref.	Title, author	Doc ID/ Version	File name	Last update	Status
[1.1]	PSA SARA Flex Flow Context Model Meijers, Patrick (PMEM)	D000886222/03	https://tc-eu.asml.com/getfile.html?D000886222-03	2020-09-01	FOR REVIEW
[1.2]	SDS Meijers, Patrick (PMEM)	D000987766/06	https://tc-eu.asml.com/getfile.html?D000987766-03	2021-06-04	UCC

Subsidiary documents

Ref.	Title, author	Doc ID	File name
[2.1]			

Reference documents

Ref.	Title, author	Doc ID/ Version	File name	Last update	Status
[3.1]	EDS of the Wafer Data Model, Tom Hoeken	D001151083/00	https://tc-eu.asml.com/getfile.html?D001151083-00	2021-12-09	Draft
[3.2]	EPS EPDS Inventory Service	D001193588-16	https://tc-eu.asml.com/getfile.html?D001193588-16	15-May-2024	UCC

- **TCE Controlling documents (auto generated, do not change)**

Ref.	Title	Author	Modified date
D000987766	SDS iACT-1	Denie van Kleef-DEKL	18-Sep-2024

- **TCE Reference documents (auto generated, do not change)**

Ref.	Title	Author	Modified date
D001795006	TDL EPS Context Model	Kamran Ahmed-KAAH	27-Nov-2025
D001795007	TDL EPS Context Model	Kamran Ahmed-KAAH	27-Nov-2025

- **TCE Documents 12NC (auto generated, do not change)**

12NC	Name	Owner	Modified date

ABBREVIATIONS

Abbreviation	Description
WDaM	Wafer Data Model
PDS	Processing Data Service
BFF	Backend For Frontends

Abbreviation	Description
cEPE	Computational Edge Placement Error
FE	Front End
UI	User Interface
ALO12c	Alignment Optimizer 12 colors
OTAS	Off Tool Application Server

Additional abbreviations can be found on the ASML Terminology Wiki

DEFINITIONS

Definition	Description

Additional abbreviations can be found on the with Findit tool

USEFUL LINKS

Link	Description

TABLE OF CONTENTS

1 INTRODUCTION 9

1.1 SCOPE 9
1.1.1 PLATFORM CORE MODEL 9
1.2 CONTEXT 12

2 OVERALL DESCRIPTION 14

2.1 CORE CONTEXT MODEL 14
2.1.1 NAMING CONVENTIONS 14
2.1.2 STEREOTYPING 15
2.1.3 USING ENUM 15
2.2 CORE MODEL - WAFER LEVEL AND UP 16
2.3 CORE MODEL – SUB-WAFER LEVEL 23
2.3.1.1 GLOSSARY OF DOMAIN CONCEPTS AT SUB-WAFER LEVEL 24
2.3.1.1.1 CONTEXT FOR FRONT-END VISUALIZATIONS 24
2.3.1.1.2 CONTEXT FOR EXPOSURE DATA 26
2.3.1.1.3 CONTEXT FOR COMPUTEJOB DATA 27
2.3.2 DOMAIN MODEL EXTENSIONS 27
2.4 CORE CONTEXT MODEL – DATABASE REPRESENTATION 28
2.4.1 CORE CONTEXT MODEL – EXPOSURE LOT LEVEL CONTEXT – DATABASE REPRESENTATION 29
2.4.2 CORE CONTEXT MODEL – EXPOSURE WAFER LEVEL CONTEXT – DATABASE REPRESENTATION 30
2.4.3 CORE CONTEXT MODEL – MEASURE LOT LEVEL CONTEXT – DATABASE REPRESENTATION 31
2.4.4 CORE CONTEXT MODEL – MEASURE WAFER LEVEL CONTEXT – DATABASE REPRESENTATION 32
2.4.5 CORE CONTEXT MODEL – LIS JOB CONTEXT – DATABASE REPRESENTATION 33
2.4.6 CORE CONTEXT MODEL – RELATION DATABASE MAPPING WITH CANONICAL PATH 34
2.5 FRONT-END SUPPORT 38
2.5.1 FRONT-END LABEL FOR CORE 38
2.5.2 ERROR! NOT A VALID LINK. FRONT-END LABEL GUIDELINES FOR DOMAIN EXTENSIONS 38

3 REQUIREMENTS 40

3.1 SOFTWARE INTERFACE REQUIREMENTS 40
3.2 FUNCTIONAL REQUIREMENTS 40
3.2.1.1 CONTEXT 40
3.3 NON-FUNCTIONAL REQUIREMENTS 42
3.4 DESIGN CONSTRAINTS 42

4 APPENDICES 43

APPENDIX A USAGE OF CONTEXT MODEL IN THE DATASET REFERENCE SPECIFICATION 43

LIST OF FIGURES

Figure 1.1 iACT Domain Model Scope 9
Figure 1.2. Wafer fab process domain concepts (D000987766) 10
Figure 1.3 Conceptual modeling of the wafer fab process [1.2] 10
Figure 1.4 Conceptual modeling of the wafer fab process – Wafer layout D000987766 11
Figure 1.5 Context Model support 12
Figure 2.1 Core context model – wafer level and up 17
Figure 2.2 Core context model – sub-wafer level 24
Figure 2.4-1 Core Context Model – Entity Relationship Diagram (Core Tables/Views) 28
Figure 2.4-2 Core Context Model – Entity Relationship Diagram (Exposure Lot Level Tables/Views) 29
Figure 2.4-3 Core Context Model – Entity Relationship Diagram (Exposure Wafer Level Tables/Views) 30
Figure 2.4-4 Core Context Model – Entity Relationship Diagram (Measure Lot Level Tables/Views) 31
Figure 2.4-5 Core Context Model – Entity Relationship Diagram (Measure Wafer Level Tables/Views) 32
Figure 2.4-6 Core Context Model – Entity Relationship Diagram (Lis Job Tables/Views) 33

LIST OF TABLES

No table of figures entries found.

1 INTRODUCTION

1.1 SCOPE

The scope of the iACT Context Model is depicted in Figure 1.1. The platform core model, aka Context Model, encapsulates the Data Platform model and the application platform core model extensions. The data platform core model describes the wafer fab process domain concepts and their relationships. The Application platform core model extensions describes elevated concepts to deliver the required platform functionality. For tenants this separation is not relevant, they should be able to access it as one platform core model, i.e the iACT Context Model.
The core of the Context Model is meant to be shared by all performance domains supported by the iACT platform. At the same time, the Context Model is designed to be extensible to cater for use cases in specific performance domains (FC extensions). Tenants extensions are grouped and owned per FC. There are no dependencies between the extensions (if this is needed, the shared model part should move down to the core model).
Domain model parts that are not FC specific and are not used by the platform itself are identified as common extensions (owned by the platform). Common extensions can be used by all FCs as context, but FC model extensions cannot have a dependency on these common extensions.

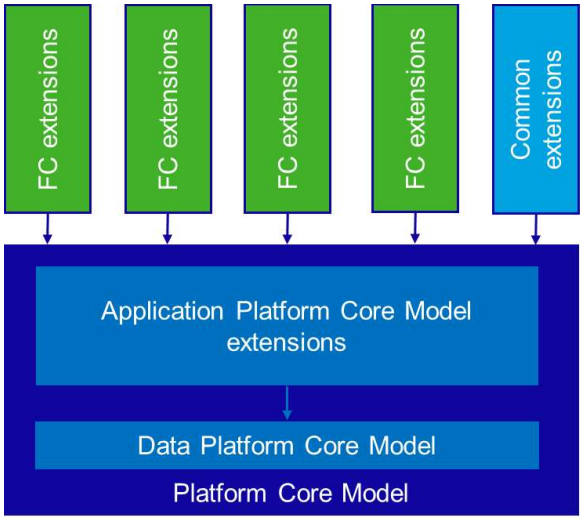


Figure 1.1 iACT Domain Model Scope

1.1.1 PLATFORM CORE MODEL

The platform core model captures the concepts in the wafer fab process domain. Figure 1.2 depicts process step, equipment, lot, wafer, layer, and process job as core concepts. The wafer fab process domain and these concepts are described in [1.1] and Section 3.2.1 of [1.2]. In summary, wafers are processed in a sequence of *process steps* of different *process step type* such as exposure, measure, deposition or etch being the relevant ones for our domain. These different step types cyclically build up *layers* as part of the stack that eventually forms the dies on the wafer substrate.
A process step, e.g. exposure for layer X, can be executed by multiple *process jobs*. The process step being the blueprint and the process job the instantiation on a certain *equipment* (machine) handling a *lot* (batch) of wafers within such a job. A *wafer lot* is a *logical group* of wafers that will go through a sequence of process jobs. The *physical grouping* by means of a carrier/foup, could be different from that[1].

[1] Although not that typical, a lot can have wafers in different foups, and a foup can contain wafers from multiple lots.

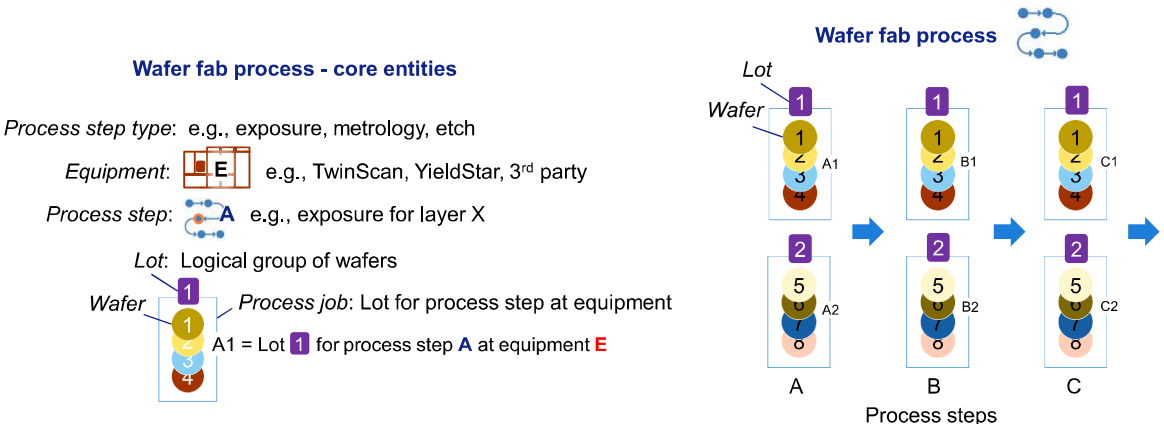


Figure 1.2. Wafer fab process domain concepts (D000987766)

The conceptual model in Figure 1.3 aims to formalize the core wafer fab domain concepts and relationships. The *core* entities and attributes represent *elevated concepts* in the sense that have explicit meaning within the iACT platform's functionality and corresponding implementation, instead of "just being data". Hence, the Core Context Model aims to support the ingestion, processing, and visualization of wafer fab process data within the iACT platform.

Next to the wafer fab process concepts, the Context Model also describes the concepts and relationships at sub-wafer level, i.e. the wafer layout hierarchy as depicted in Figure 1.4.

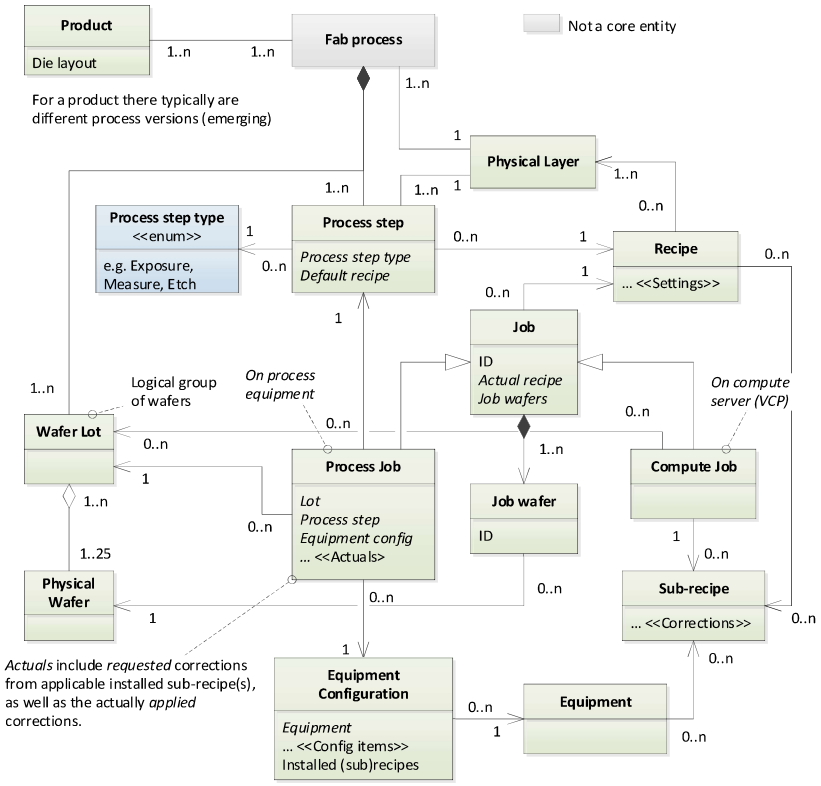


Figure 1.3 Conceptual modeling of the wafer fab process [1.2]

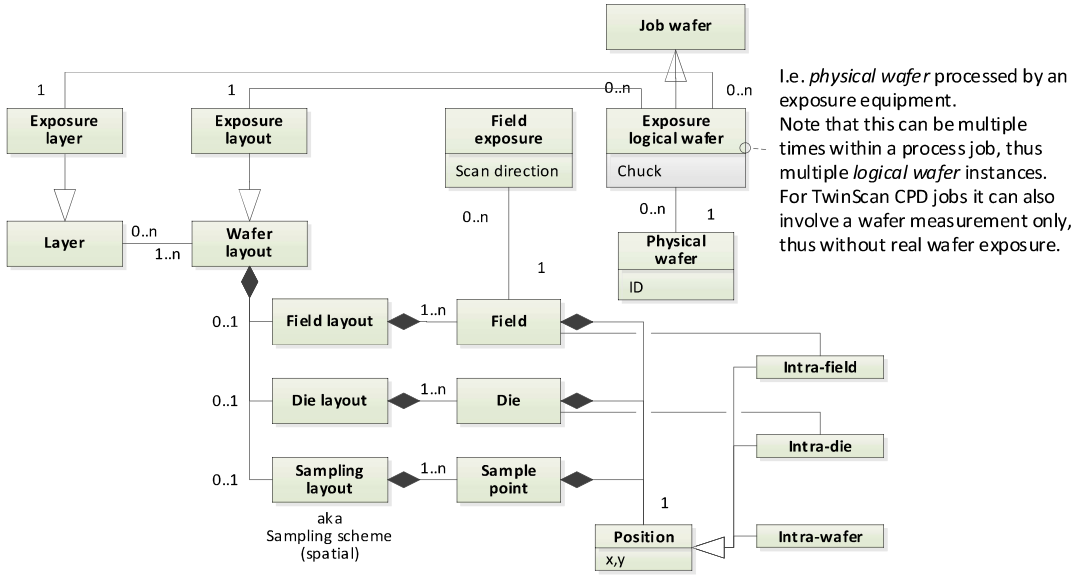


Figure 1.4 Conceptual modeling of the wafer fab process – Wafer layout D000987766

1.2 CONTEXT

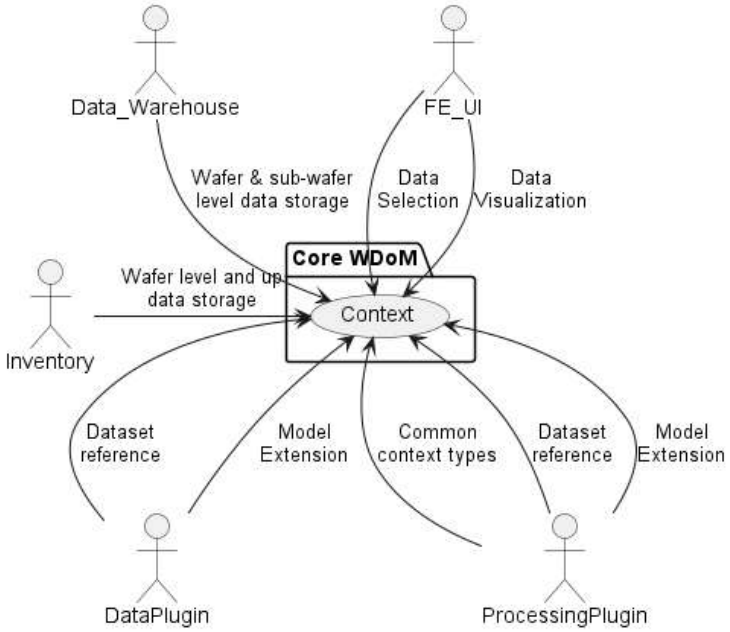


Figure 1.5 Context Model support

The table below summarized the concerns supported by the context model

User	Aspect	Description
Inventory	Wafer level and up storage	- Context at wafer level or above is stored in the LithoInSight Analytics relational database (PostgreSQL) and VIEWs on top of those relational tables expresses the Contexts as close as possible to Class Diagram in EPDS. There are still some contexts which can't be concretely mapped to LithoInSight Analytics Database as defined in Figure 2.1 Core context model – wafer level and up, hence an explanation and mapping for such context is given in section 2.4.6
Data Warehouse	Wafer & sub-wafer level data storage	- Context information is required in the data warehouse to store wafer up and sub-wafer level data. This is realized using the context columns of the persisted datasets in the data warehouse. Note that context columns can be for wafer level and up, and sub-wafer level. For the latter, the dataset use the Context Model's entities and relationships at sub-wafer level. This specification is defined by the data plugin corresponding dataset references.
Data Plugin	Dataset references	- Data plugins read performance domain specific data from a backend system, transform the data if needed, and write it subsequently into the data warehouse as one or more datasets. The name of each column (context) in a dataset is defined as a path from an anchor point in the Context Model to an entity-attribute leaf, e.g. `exposureprocessjob_waferexposureprocessjob_wafer_keywaferid` for the canonical WaferId. Conceptually, the anchor point can be any job type such as ComputeJob. See example in see Appendix 0. Because the constructed logical column names are also used as column names in Spark (to avoid unnecessary mappings), the format is limited to lowercase characters and underscores due to Spark storage related limitations.

		Note that this also implies that backward incompatible changes in Context Model will require (customer) data migration(or re-ingestion) and therefore need to be prevented to the maximum extend. • See OI08: Column naming will be redefined in future revisions.
	Model Extension	- The Context Model shall be extendable. Data plugins use the Context Model as baseline to add their domain specific entities, attributes, and relationships at point, sub-wafer, and wafer-up level. Note that the extension support is not covered by the current Context Model version.
Processing Plugin	Common context types	- The flows implemented in a processing plugin depend primarily on the context types defined in its corresponding domain dataset reference(s) and the common concept and context types in the Context Model.
	Dataset reference	- The outputs of processing plugins are persisted in the data warehouse as new datasets.
	Model Extension	- The Context Model shall be extendable at runtime. Processing plugins use the Context Model as baseline to add their domain specific entities, attributes, and relationships at point, sub-wafer, and wafer-up level. Note that the runtime extension support is not covered by the current Context Model version.
FE UI	Data Selection	The FE UI support two main data selection phases, Coarse and Fine. Both of these are implicitly based on Context Model concepts. Coarse selection: - The coarse data search pane provides filter criterion based on Context Model wafer-level and up context types, and when configured by extensions, wafer-level and up context for specific domains after the extensibility mechanism is in place. - The columns in the data record table (coarse search results) map to wafer-level up entities in the Context Model. However, there isn't any explicit logic that couple the relationships or hierarchies in the Context Model with the table results' layout Fine selection: - Context Model context types and the corresponding context values are used in the view configuration panes
	Data Visualization	Data visualization are realized by views that contain single or multiple plots. Context Model core concepts are used to classify, configure, and trellised visualization plots. - Plot types map Context Model core entities like: wafer, field, die, etc. - Plot configurations rely on Context Model core concepts for aggregation levels, e.g. field, die, and wafer point

1
2

2 OVERALL DESCRIPTION

2.1 CORE CONTEXT MODEL

This section specifies the realization of the conceptual model, provided in Figure 1.3, considering the current fab automation(reflected in LIS/LANA) and the aim to have a lean, essential, and extensible core domain model. For readability purpose, the core context model is divided in two class diagrams,

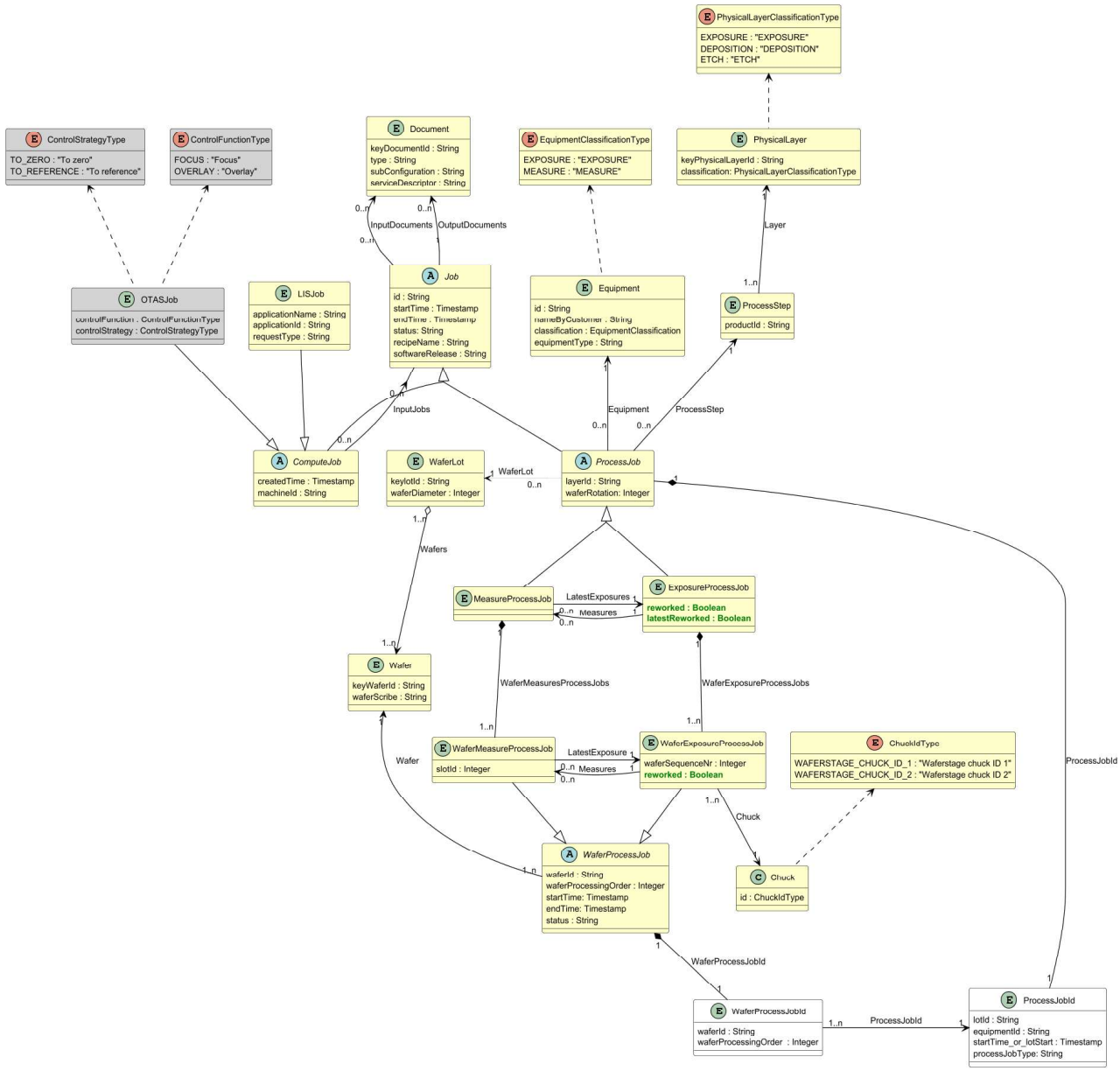


Figure 2.1 and Figure 2.2. The link between these two diagrams is compose by the WaferLot and Wafer entities.

2.1.1 NAMING CONVENTIONS

Entity ID's name shall distinguish between canonical and associating IDs:

- **Canonical IDs are attributes** of so-called canonical entities that represent real “physical” things in a unique unambiguous way. E.g. the real physical wafer or physical layer (stack state). These are typically constructed from extracting above mentioned context information and translated by means of an association model based on customer specific conventions for naming the association IDs [1.2]. Canonical ID names shall contain the prefix “key” and postfix “Id”, e.g. *keyLayerId*. Canonical IDs are required for core entities, e.g. layer, wafer lot, recipe, and wafer.

  For consistency, domain extensions shall apply this naming convention for domain specific entities that represent a real physical thing. In addition, for sake of simplicity, any Id attribute produced by the LANA backend based on normalization logic should use the prefix “Key”.

- **Associating IDs are attributes** that identify other entities based on a value that is ingested from a data source, typically a job report. E.g., layer ID, lot ID, wafer ID, chuck ID, etc. as extracted from the corresponding ADEL XML elements. Note that these do not need to be unique and require additional context information for correct interpretation [1.2]. An association ID in an entity reflect an implicit relationship to the target entity. However, a so called normalization process is used to map an associating ID to the canonical ID, and hence to the target entity. Associating ID names shall only contain the entity name and the postfix “Id”, e.g. *waferId*.

  Note that other identification attributes can be added by the ingestion process for internal purposes, e.g. WaferProcessJobId and ProcessJobId.

- Avoid context redundancy in attribute names, i.e. the entity name shall not be reused in its attributes names. For example, using JobId in the Job entity is redundant. Instead, use only Id as attribute name.

2.1.2 STEREOTYPING

- Abstract entities are annotated with an  . These entities are abstract definitions to capture commonality, i.e. common attributes across related concepts. ‘

- Final entities are annotated with an  . These entities are actual domain concepts or specializations of abstract entities.

- Enumeration types are annotated with an  . Enumeration types are introduced to typify domain concepts. Hence, the naming convention for enumerations is conceptNameType, where conceptName is the domain concept, e.g. chuckIdType.

- Combined key entities, in white, are platform computed entities and are added to the domain model to enable the unique identification of ProcessJob and WaferProcessJobs across the layers in the iACT platform.

2.1.3 USING ENUM

How enum values are to be used and stored with in platform.

- All platform tenants e.g. Inventory and Plugins should use the platform provided enum *values* instead of enum *constant* for storage. Such as for ChuckIdType we have “WAFERSTAGE_CHUCK_ID_1” as constant and “Waferstage chuck ID 1” as value. We kept it that way to reduce migration impact on existing plugin usages where deviation from above mentioned style would cause Data Migration issue.

- If needed a mapping to convert extracted values back to platform supported value is to be added in tenants.

- All existing enum are also restructured to make it consistent without any impact on current usages.

- As a baseline any new enum addition in Context Model should follow the same approach of using source as value provider for platform.

- We have two different usages category in Context Model w.r.t to enum i.e. the one driven from ADEL source such as ChuckIdType and ControlStrategy and the others we created to distinguish core entities such as EquipmentClassificationType where source is not ADEL, for the former ADEL must be the source of truth unless we have any valid/strong reason to deviate and for the latter we use UPPER SNAKE_CASE as enum constant and value.

- **Disclaimer:** Any deviation in stored values wouldn't be supported by platform for cross domain analysis use cases. E.g. If a plugin is storing ChuckIdType as 'CHUCK_1' which deviates Context Model hence no support for cross domain analysis.

2.2 CORE MODEL - WAFER LEVEL AND UP

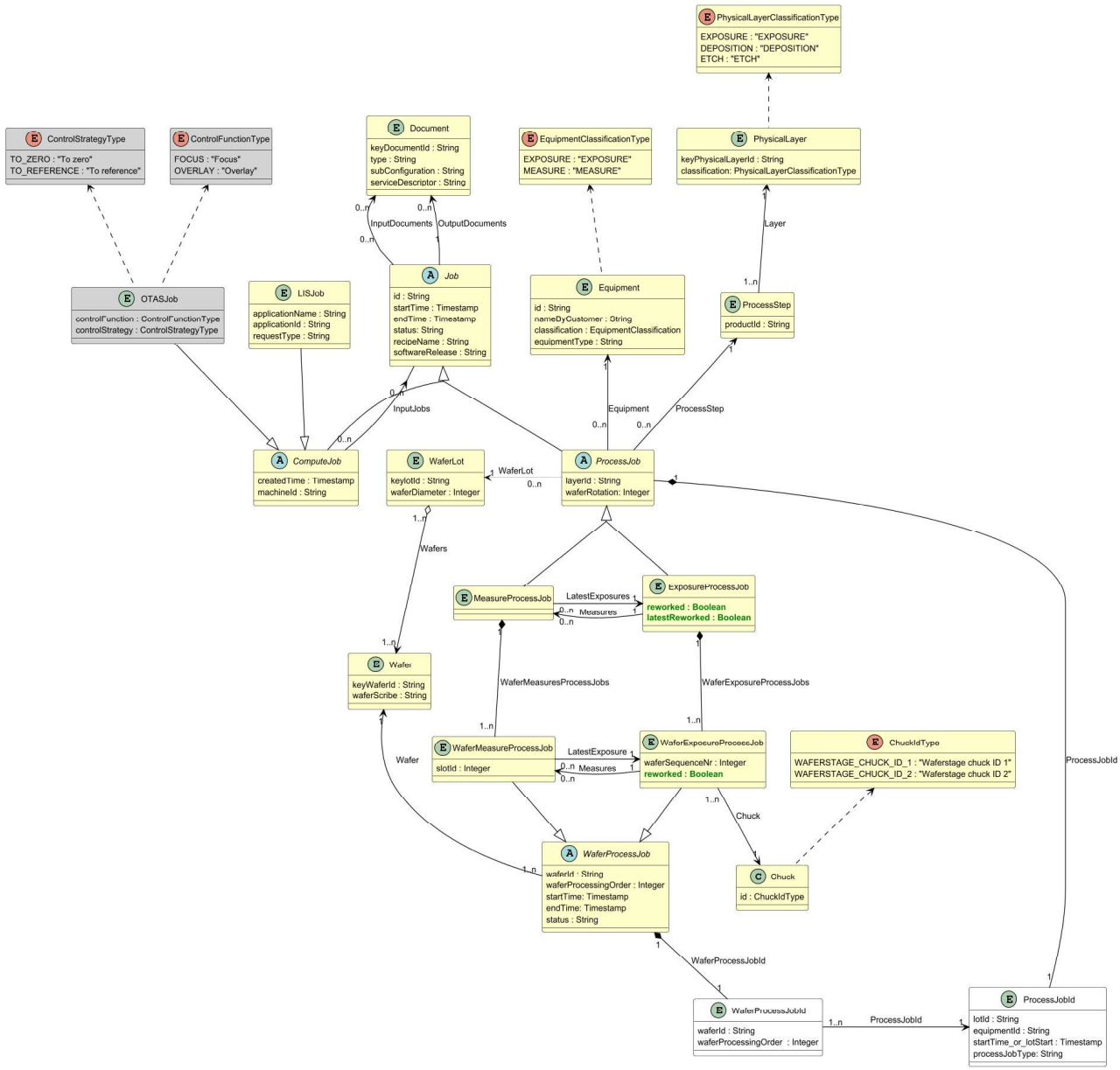


Figure 2.1 Core context model – wafer level and up

Glossary of domain concepts at wafer level and up
Note that the attributes that were removed or renamed with respect to version 1.0.0 are kept for review purpose as ~~removed~~. To see removed items from previous versions, see previous TCE revisions of the document.

Document

Description	ADELdocument (currently, later logfiles, binary files).
Association	Input and/or output documents of a Job.
Attributes	
KeyDocumentId	Identification: key if available (e.g. from DocumentStore). Note: this is not the Id in the ADEL documents, but it is normalized by the LANA backend, hence it uses the prefix “Key”.
type	Type of the document (e.g. ‘ADELmetrology’).
subConfiguration	The sub configuration (eg: prod, beta). Note that in case of LIS this does not make sense. Prod and beta share the same document database.
serviceDescriptor	The service descriptor (OTAS, ObjectStore)

Note: For future cases: add context to Document to e.g. distinguish between different fingerprints created (via data plugin).

Job

Description	Job
Generalization	Common attributes of ProcessJob and ComputeJob .
Attributes	
id	Identification of the job (e.g. ‘2A-N96Y08.L9-NFC009161_20120427_011427’). The value depends on the Job specialization, see description of ProcessJobs and ComputeJobs
startTime	Time when the job started (e.g. ‘2012-04-27T01:14:27.503000+08:00’).
endTime	Time when the job ended (e.g. ‘2012-04-27T01:18:15.850000+08:00’) .
status	Status of the job with possible aggregation per wafer (as e.g. in modeling job).
recipeName	Name of the job recipe, in a human readable format. At the moment it matches the path name for the recipe. Note that in the case of processJobs it is ingested by the platform using ADELler (LotSettings/ExposureRecipeId) or ADELMetrology(Input/RecipeInformation/Name). In the computeJobs, it is ingested by the corresponding data plugin.
softwareRelease	Release under which the job ran (e.g. ‘R52’, collection of installed packages).
Relationships	
InputDocuments	Subset of input documents, used by the job.
OutputDocuments	Subset of output documents, created by the job.

ProcessJob

Description	Job executed on a Twinscan or Yieldstar or any other equipment.
Generalization	Common attributes for ExposureProcessJob and MeasureProcessJob .

Attributes	
layerId	Association Id of the layer associated to this processJob
waferRotation	Wafer rotation applied to all the wafer processed with this process job. Note, that wafer rotation is a synonym for WaferLoadOrientation in the case of Yieldstart processJobs
Relationships	
ProcessStep	Definition of the corresponding process step.
WaferLot	Definition of the associated lot.
Equipment	The equipment where the job ran on.
ProcessJobId	Combined key entity for processJob
	- lotId - equipmentId - startTime - processJobType: required to link the corresponding Expose or Measure processJob at the WDaM side.

ComputeJob

Description	Job that did not ran on equipment, but on platforms like LIS/Spark/OTAS/..
Generalization	Common attributes of Job
Attributes	
createdTime	Time the job was created (e.g. for performance analysis).
machineId	Identification of the machine (not the EquipmentType). For VCP the instance id.
Relationships	
InputJobs	List of input jobs (i.e. chaining).

All the above relationships can be solved by navigating the context model. Such navigation will be enabled by Inventory APIs

LISJob see OI09

Description	LIS Job ingested in the iACT platform
Generalization	Common attributes of ComputeJob and Job
Attributes	
applicationName	Job name (e.g. 'overlay estimation') / functional execution environment
applicationId	The application id for the LIS job, e.g. alignment, ep. Note that applicationId can be similar to applicationName in some cases.
requestType	The request type for the LIS job, e.g. monitor, kpiMonitor

OTASJob

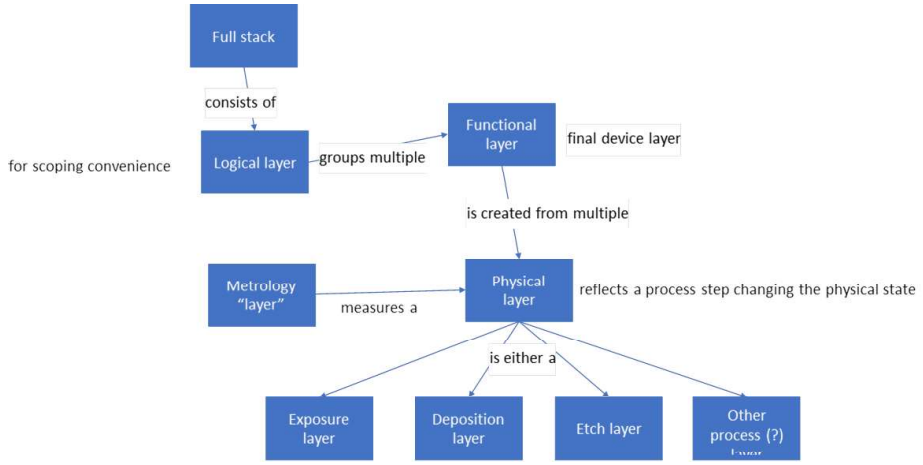
Description	OTAS Job ingested in the iACT platform (OTAS is a different tool than LIS), this addition is temporary and grey entities are for differentiation. See OI14

Generalization	Common attributes of ComputeJob and Job
Attributes	
controlFunction	'Focus' or 'Overlay', see ControlFunctionType. Control function of the job (taken from schema documentation).
controlStrategy	'To Zero' or 'To reference', see ControlStrategyType. Control strategy of the job (taken from schema documentation)

ProcessStep

Description	The step in the process to produce (part of) a product on a layer using a recipe
Attributes	
productId	Identification of the product to be created (e.g. 'TMDK68').
Relationships	
Layer	Associated layer for the process step

PhysicalLayer

Description	It reflects the layer identification associated to the process that actually change the physical state of the wafer. Note that there are several concepts around layer. The diagram below aim to clarify this and the place for PhysicalLayer. 
Attributes	
keyPhysicalLayerId	Normalized layer Id, e.g. 'LAYER_AP_0001'
classification	Physical layer classification, e.g. Exposure, Deposition, Etch. These are the actuation process steps that change the physical state of the wafer. Note, that there are more actuation steps, e.g. develop and cleaning, but only few are considered based on the coming use cases. See PhysicalLayerClassificationType for the current values.

ExposureProcessJob

Description	Exposure job ran on a Twinscan
Attributes	
reworked	Indicates If current exposure lot is reworked or not, by default 'false' for every lot until it is reworked.
latestReworked	Indicating if current exposure lot is latest reworked lot. E.g. Lot#1 Exposed and then reworked multiple times subsequently as Lot#1.1, Lot#1.2 and Lot#1.3 so

	the Lot#1.3 was never reworked hence the latest reworked lot is only Lot#1.2. (names are just given for understanding of sequence, in real world all reworked lots would be with same LotId)
Associations	
WaferExposureProcessJobs	Composition of sub-jobs for all wafers in the associated lot.
Measures	List of corresponding measurement jobs (explicitly enforced relation during ingestion by Inventory).
Job.Id	For ExposureProcessJob, the jobId is extracted from the LotId field in the ADELIer

MeasureProcessJob

Description	Measure job ran on inspection equipment
Attributes	
Associations	
WaferMeasureProcessJobs	Composition of sub-jobs of all wafers in the associated lot.
~~latestExposureProcessJob~~ LatestExposures	Corresponding latest exposure jobs (explicitly enforced relation during ingestion by Inventory).

WaferProcessJob

Description	Wafer process job.
Generalization	Common attributes of WaferExposureProcessJob and WaferMeasureProcessJob
Attributes	
waferId	Association Id of the wafer the job ran for.
waferProcessingOrder	The order in the which the given machine processes the wafer (• Value is extracted from ADELIer (waferProcessingOrder) for exposure jobs • Value is calculated by LANA for measure jobs
startTime	The start time of this job
endTime	The end time of this job
status	Indication about the final processing status of the wafer, e.g. rejected, accepted, etc.
Associations	
ProcessJobId	Id of the originating exposure/measure process job.
WaferProcessJobId	Combined key entity for interactions with WDaM and not visible by domain plugins
	- waferId - waferProcessingOrder: The order in the which the given machine process the wafer (see above)

WaferExposureProcessJob

Description	Wafer level ExposureProcessJob
Attributes	
waferSequenceNr	Wafer Sequence Number from 1-25 in a Lot.

reworked	Indicates If a wafer is reworked/re-exposed or not, by default 'false' for every wafer until it is reworked.
Associations	
measures	List of corresponding wafer measurement jobs

WaferMeasureProcessJob

Description	Wafer level MeasureProcessJob
Attributes	
slotID	Slot location of the wafer within the carrier when placed at the metrology tool.
latestExposure	Corresponding latest exposure job (explicitly enforced relation).

Equipment

Description	Equipment on which process jobs run.
Attributes	
id	ASML identification of the equipment as registered in the Equipment Manager(EQMA).
nameByCustomer	The identification given by the customer owning the equipment, as described in the ADEL path: Header\ MachineCustomerName.
classification	'EXPOSURE' or 'MEASURE', see EquipmentClassificationType.
equipmentType	Specific equipment type, e.g. NXT* / NXE* or ~~Yieldstar~~ S-375.

WaferLot

Description	A wafer lot is a logical collection of wafers. Process jobs run per wafer lot, though still selecting a subset of the wafers within a lot to actually process. Compute jobs can run on one or more wafer lots and subsets of wafers from that.
Attributes	
keyLotId	The normalized Id for the lot using the information found in the ADEL document of the corresponding processJob. Note that physical key for lot is rarely available or to be derived, E.g. for ExposureProcessJob, the KeylotId can be derived from the LotId value in the ADELler, but this is not the actual lotId
waferDiameter	Wafer size applicable to all the wafers associated to this lot
Associations	
Wafers	List of associated wafers to this lot

Wafer

Description	Wafer
Attributes	
keyWaferId	Identification of the wafer.
waferScribe	Marker on wafer with physical wafer identification.

Chuck

Description	Chuck, only relevant in exposure process jobs.
Attributes	
id	Enum values in sync with ADELler schema **Disclaimer:** All platform tenants e.g. Inventory and Plugins should use the platform provided value for storage. If needed a mapping to convert extracted values back to platform supported value is to be added in tenants. Any deviation in stored values wouldn't be supported by platform for cross domain analysis use cases Since this specific enum is bound to ExposureProcessJob which results in creation of ADELler hence ADEL was used as source, that could vary for other enums. - Waferstage chuck ID 1 - Waferstage chuck ID 2 Note that the Scanner model has CHUCK_1 and CHUCK_2 as values, but this can be unclear in other scenarios like reticle stage chuck.

2.3 CORE MODEL – SUB-WAFER LEVEL

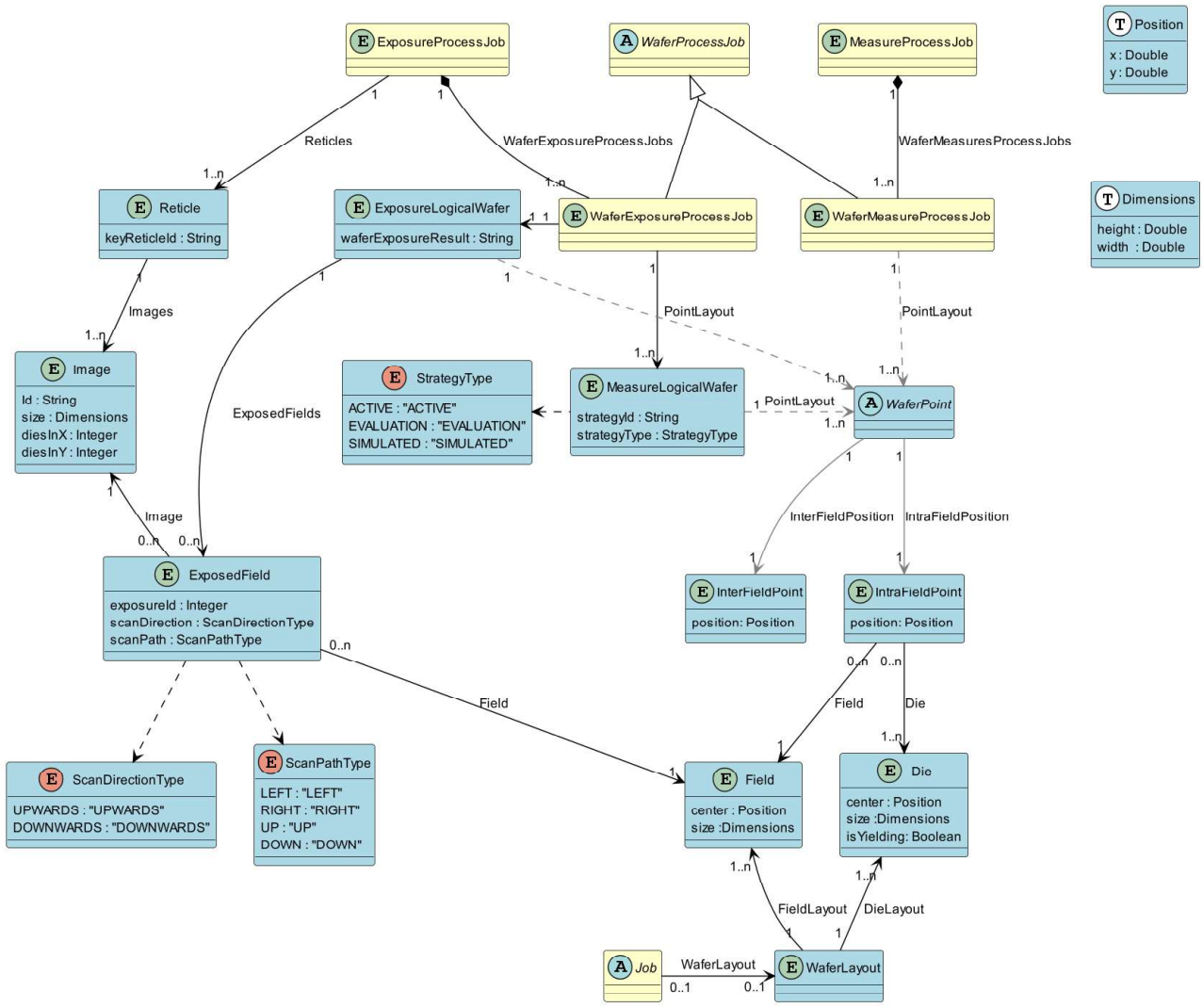


Figure 2.2 Core context model – sub-wafer level

Yellow entities refer to the core model wafer level and up. Blue entities refer to the sub-wafer level and do not reside in the inventory.

2.3.1.1 GLOSSARY OF DOMAIN CONCEPTS AT SUB-WAFER LEVEL

2.3.1.1.1 CONTEXT FOR FRONT-END VISUALIZATIONS

WaferLayout

Description	This is a platform concept to bundle wafer layouts. This concept enables the FE to query commonly used layouts within wafer plot panes. The current approach is that domain data plugins are responsible to populate the associated layouts.
Attributes	
Associations	
fieldLayout	List of fields associated to the main ProcessJob in the ingested data (decided by the data plugin). Note that multiple field layouts can be distinguished by their size (height and width).
dieLayout	A scatter of dies, which is regardless of the image configuration and field layout, on the wafer. The die layout is associated to the main (Process)Job in the ingested data, decided by the data plugin.

Field

Description	The core representation to specify a field on the wafer. Note that this entity can be used even if the exposure information is not available, e.g. Yieldstart measurements.
Attributes	
center	X and Y position of the field center with respect to the center of the wafer.
size.width	Width of the field. Most of the times, this value is the same as the associated image width.
size.height	Height of the field. Most of the times, this value is the same as the associated image height.
Associations	
~~Image~~	~~The reticle image that was used to expose the field~~

Die

Description	The core representation to specify a die on the wafer. Note that this entity can be used even if the exposure information is not available, e.g. LIS jobs.
Attributes	
center	X and Y position of the die center with respect to the center of the wafer.
size.width	Width of the die.
size.height	Height of the die.
Associations	

PointLayout - set of WaferPoint

Description	This is an abstract platform concept to capture the common pattern to persist point-level data per wafer. The FE relies on this pattern to query and plot point level data from different domains (multi-domain). Note that domain data plugins are responsible to specialize this abstract concept according to their requirements and it can be associated to concepts like measureLogicalWafer, waferMeasureProcessJob, waferComputeJob and ExposureLogicalWafer. Connection from ExposureLogicWafer was initialed for IMO3 (leveling) but it make sense for other domains too hence added to core model.
Attributes	
Associations	
IntraFieldPosition	The intrafield position of the associated point on the wafer. Each intrafield point must be associated to a Field, hence its position is in the field coordinate system. Additionally, intrafield points can also be associated to one or more dies.
InterFieldPosition	The interfield position of the associated point on the wafer. Each point in this list is independent of the exposure layouts, hence its position is in the wafer coordinate system.

Position

Description	Common base concept to specify the X and Y coordinates of a point on the surface of the wafer, sensors, wafer stage, etc.
Attributes	
X	X coordinate value
Y	Y coordinate value
Associations	
	The Position concept is used in the context model to represent the center of a field, die, and wafer points (intrafield and interfiled). **Note** that the association used in the model is as value object to ease readability, and it is not intended to be used as complex type yet. Hence, context paths using Position, shall be translated on two separated paths each ending in position_X and position_Y.

Dimensions

Description	Common base concept to specify the height and width of an 2D object on the surface of the wafer, sensors, wafer stage, etc.
Attributes	
height	height value
width	width value
Associations	
	The Dimensions concept is used in the context model to represent the dimensions of the field, image and die. **Note** that the association used in the model is as value object to ease readability, and it is not intended to be used as complex type yet. Hence, context paths using Dimensions, shall be translated on two separated paths each ending in dimensions_height and dimensions_width.

2.3.1.1.2 CONTEXT FOR EXPOSURE DATA

ExposureLogicalWafer

Description	Provides context for the wafer-level activities at the expose side of the scanner. It is introduced to match the Twinscan core model. Note that further attributes can be added by domain plugins.
Attributes	
waferExposureResult	Exposure result of the wafer. Current possible values are: FullyExposed, PartiallyExposed, NotExposed.
Associations	
ExposedFields	The list of fields that were exposed on the wafer in the context of the associated waferExposureProcessJob.

ExposedField

Description	Provides context attributes of a field exposed on a wafer using an image reticle. Note that this information will be only available for use cases that involve ExposureProcessJob data.
Attributes	
exposureId	Unique identification of the exposure within the image. It can be used to relate exposure data across the different exposureprocessjob documents.
scanDirection	The direction of the scan during exposure. See the possible values on the enum ScanDirectionType
scanPath	The path of the scan during exposure. See the possible values on the enum ScanPathType
Associations	
Field	The field information, i.e. position, of the exposed field.
Image	The image that was used to expose the field.

Reticle

Description	It represents one of the reticles that was used within the ExposureProcessJob
Attributes	
keyReticleId	The canonical id of the reticle
Associations	
Images	The set of images defined in the reticle

Image

Description	It represents one of the images that was used within the ExposureProcessJob
Attributes	
id	Identification of the image
size	Width and height of the image
diesInX diesInY	A grid configuration of the dies per image. Note that this configuration can be used by domain plugins to generate the Die layout for the associated (Process)Job.

Associations	

MeasureLogicalWafer

Description	Provides context for the wafer-level activities at the measure side of the scanner. It is introduced to match the Twinscan core model. Note that further attributes can be added by domain plugins.
Attributes	
strategyId	Identification of the strategy, formally known as alignment strategy, but used to bundle all the wafer measure side phases, i.e. alignment and leveling.
strategyType	Possible values are ACTIVE and EVALUATION in the context of exposure jobs, and SIMULATION for processing data scenarios implemented in iACT.
Associations	
pointLayout	A domain specific set of points measured in the context of the given strategy.

2.3.1.1.3 CONTEXT FOR COMPUTEJOB DATA

WaferComputeJob

This is put on hold until a structural solution is in place to replace the containment for the cEPE extension, see D001214801

~~Description~~	~~Provides context for virtual wafers defined by computeJobs. For instance, it provides context for aggregated/computed data from an actual waferProcessJobs. In contrast to WaferProcessJobs, this wafer entity depends on domain knowledge, hence it is in the scope of the sub-wafer level data ingested by data plugins.~~
~~Attributes~~	
	~~To be defined as part of the cEPE use case. See OI13~~
~~Associations~~	
~~waferProcessJob~~	~~The associated waferProcessJob~~

2.3.2 DOMAIN MODEL EXTENSIONS

Domain model extensions are documented on their corresponding domain SEG documents:

- Overlay extension see D001070766
- Wafer alignment extension see D001247120
- cEPE extension see D001214801

The sources for the current context models and extensions can be found in: iACT Context Model - Bitbucket PRD EID (asml.com)

2.4 CORE CONTEXT MODEL – DATABASE REPRESENTATION

A context model describes the semantics of its contexts and their logical associations. The contexts can be stored in various ways depending on usage and tools (e.g., RDBMS). These representation differ fundamentally. The following Entity Relation Diagram below is a new interface for plugin and application to query contextual data from Analytics Database. It is not a complete view of Analytics Database but only the tables/views involved in making Core Context accessible, further details on each Core Context Model entity is following this section. Each Table in details section is just showing relevant columns/fields while the real Tables contain much more information than what is depicted in Core Context Model.

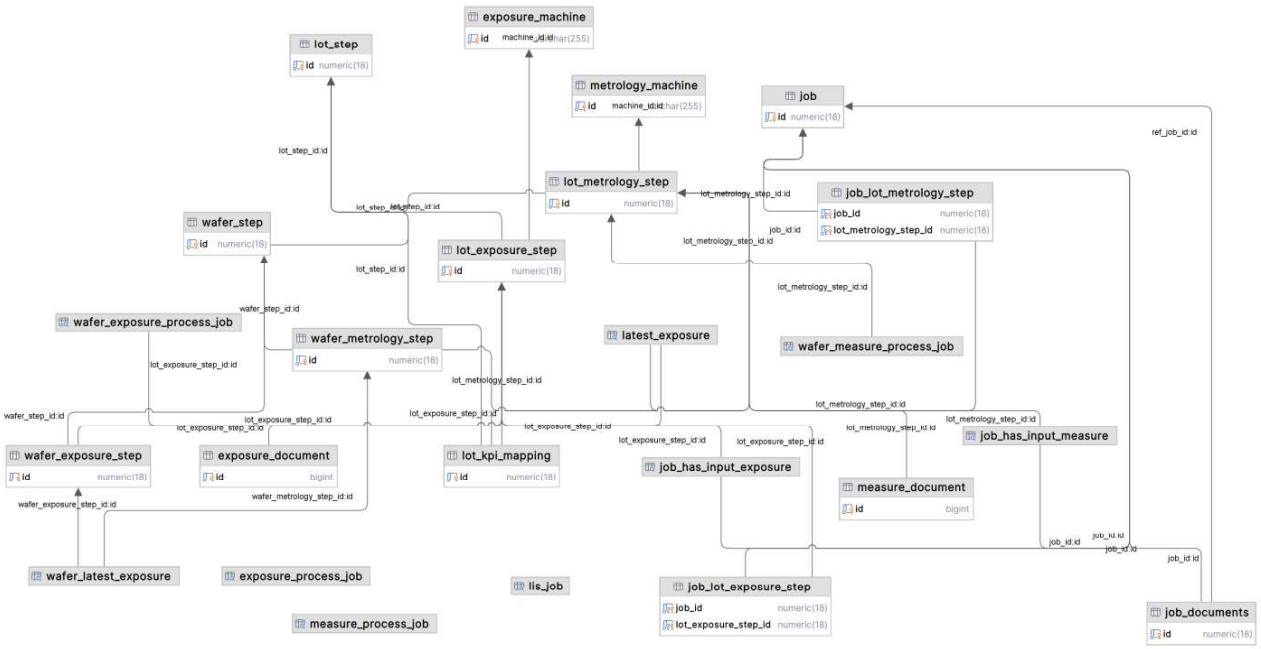


Figure 2.4-1 Core Context Model – Entity Relationship Diagram (Core Tables/Views)

2.4.1 CORE CONTEXT MODEL – EXPOSURE LOT LEVEL CONTEXT – DATABASE REPRESENTATION

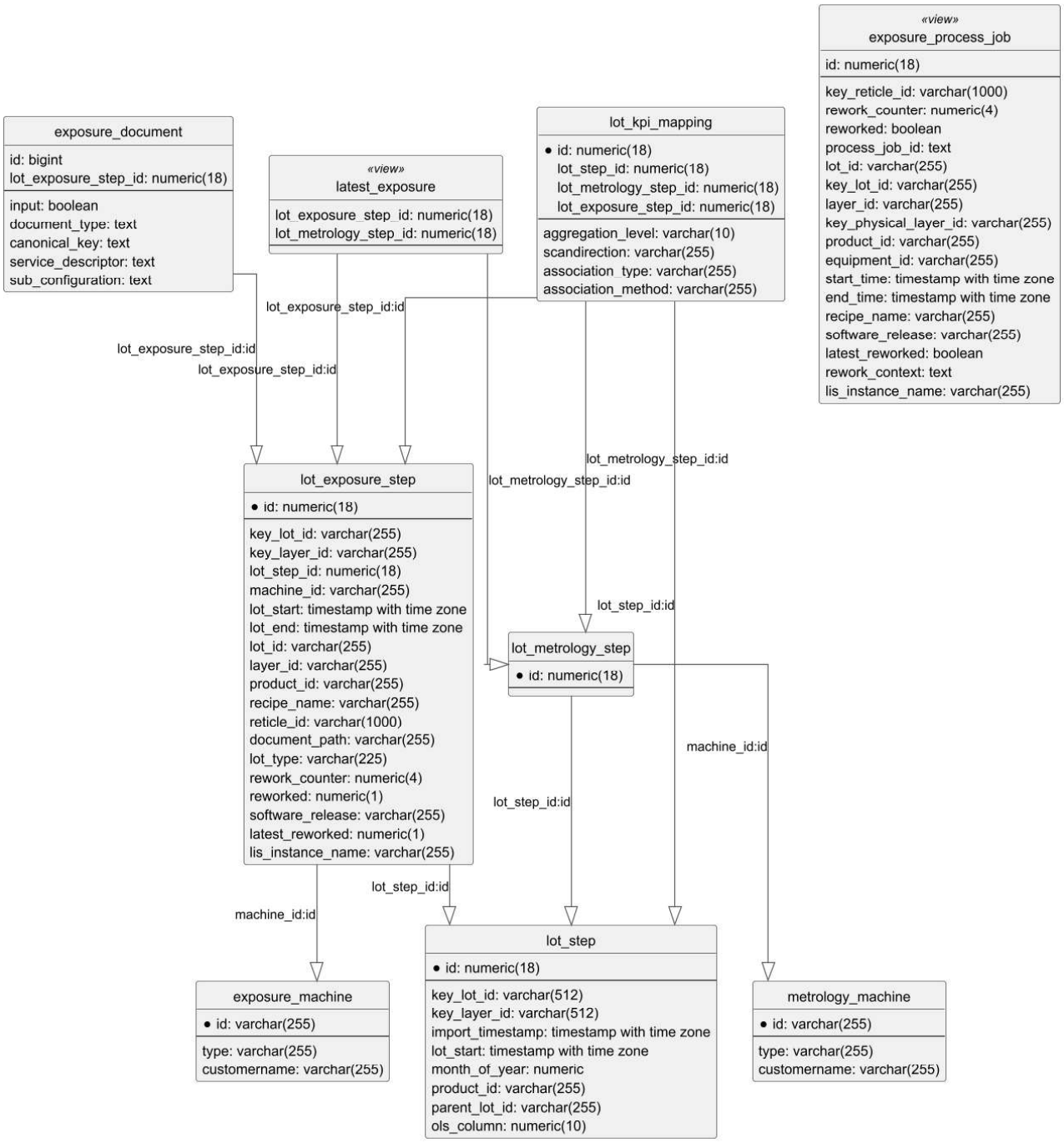


Figure 2.4-2 Core Context Model – Entity Relationship Diagram (Exposure Lot Level Tables/Views)

2.4.2 Core Context Model – Exposure Wafer Level Context – Database Representation

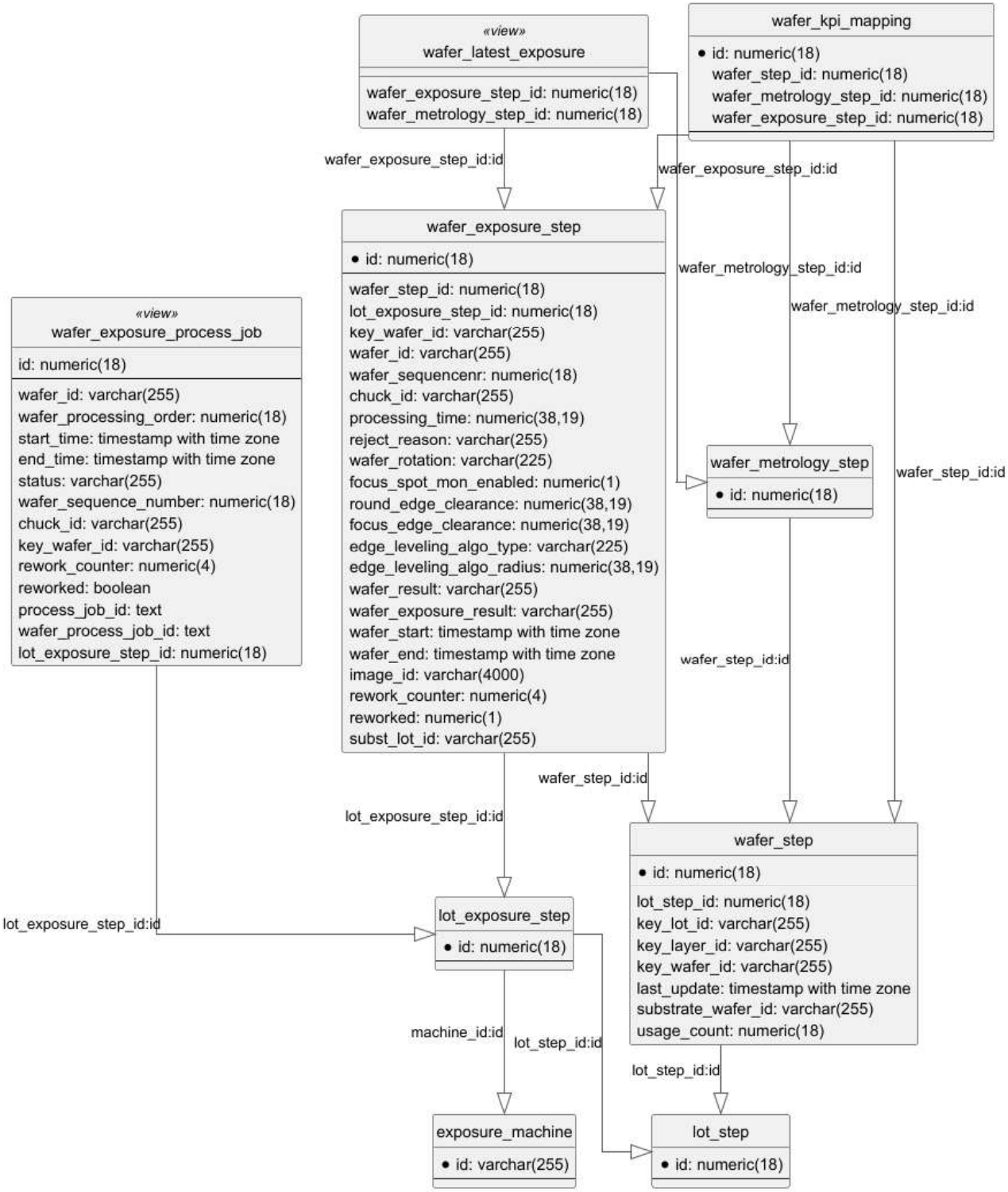


Figure 2.4-3 Core Context Model – Entity Relationship Diagram (Exposure Wafer Level Tables/Views)

2.4.3 CORE CONTEXT MODEL – MEASURE LOT LEVEL CONTEXT – DATABASE REPRESENTATION

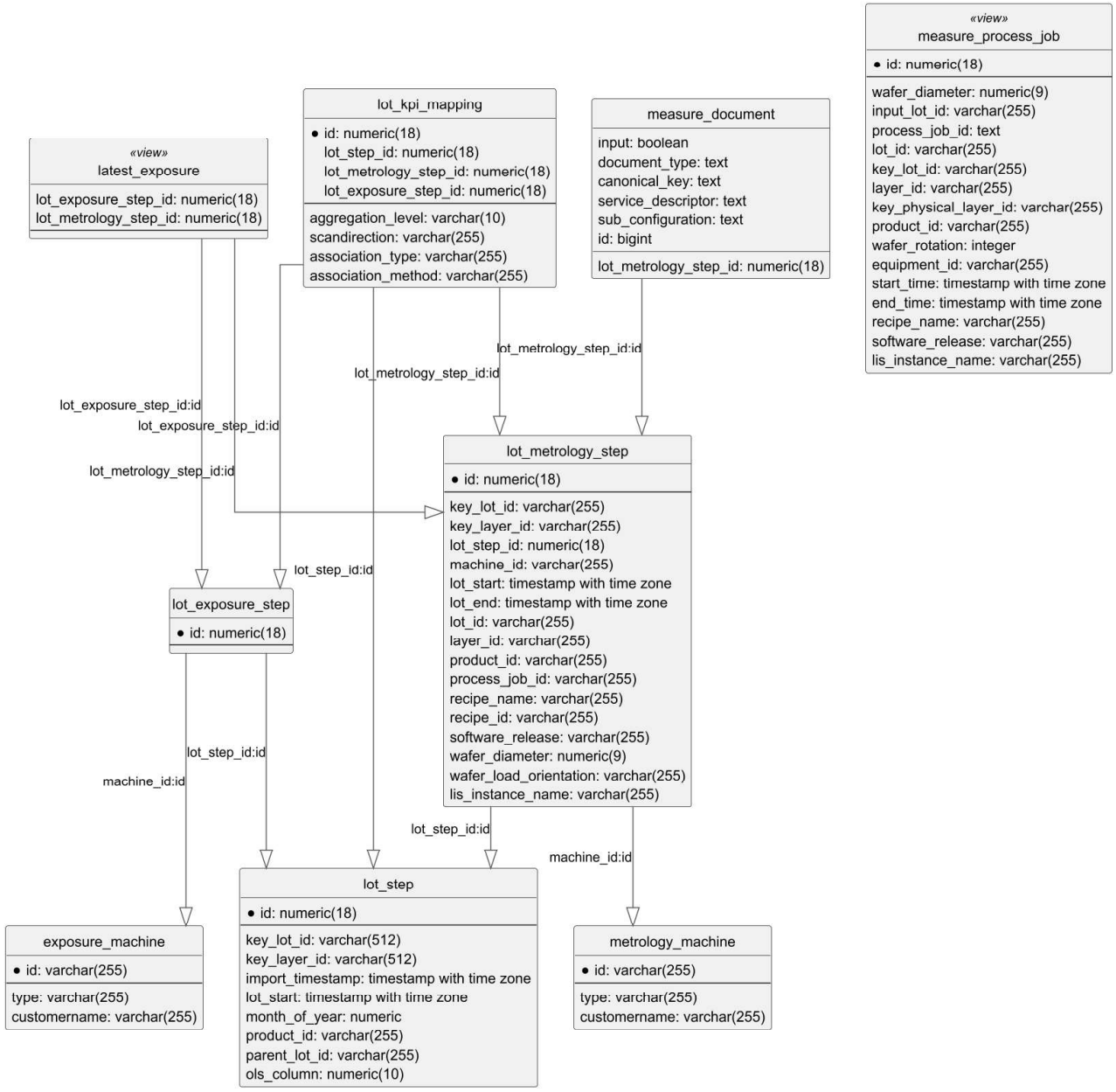


Figure 2.4-4 Core Context Model – Entity Relationship Diagram (Measure Lot Level Tables/Views)

2.4.4 CORE CONTEXT MODEL – MEASURE WAFER LEVEL CONTEXT – DATABASE REPRESENTATION

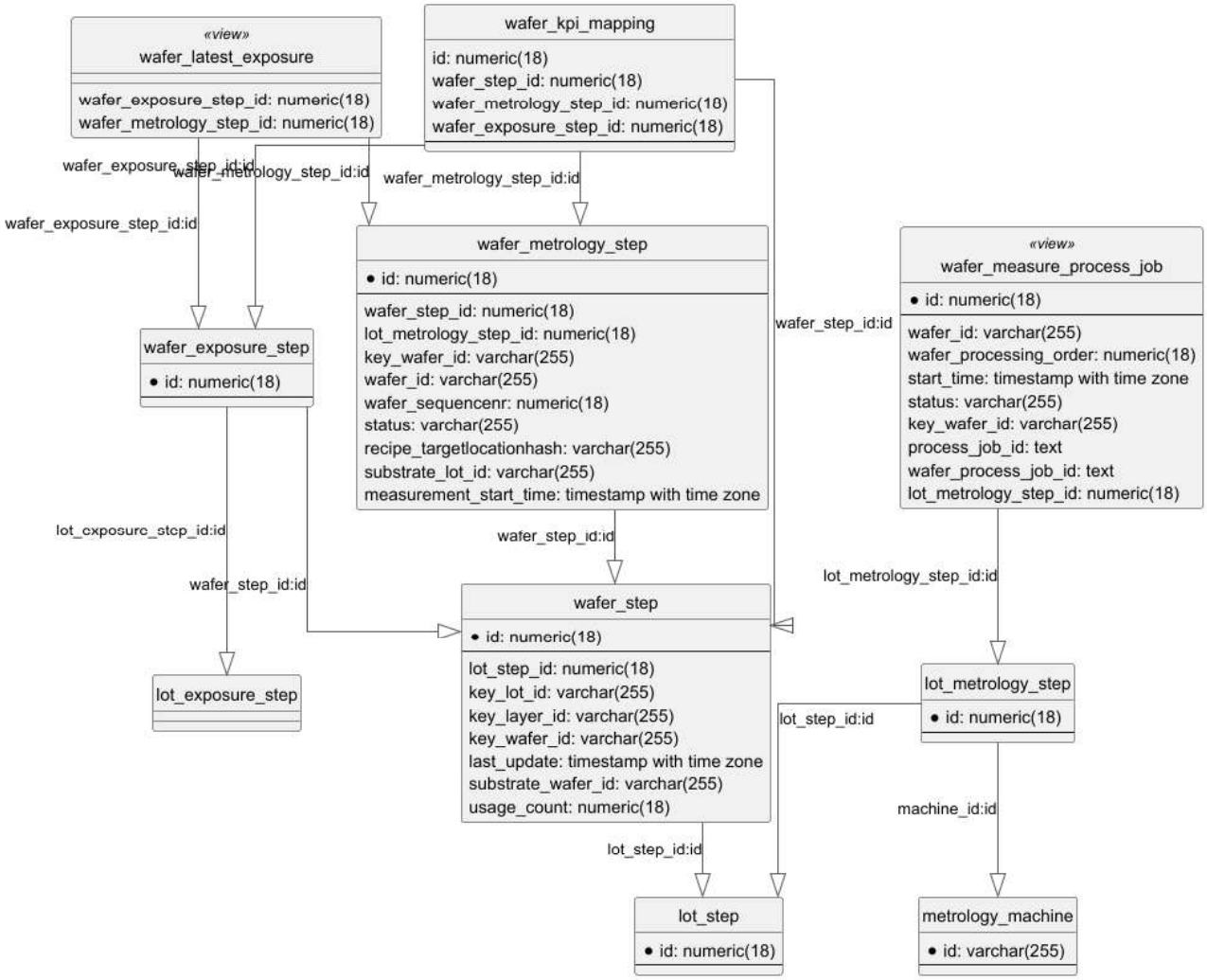


Figure 2.4-5 Core Context Model – Entity Relationship Diagram (Measure Wafer Level Tables/Views)

2.4.5 CORE CONTEXT MODEL – LIS JOB CONTEXT – DATABASE REPRESENTATION

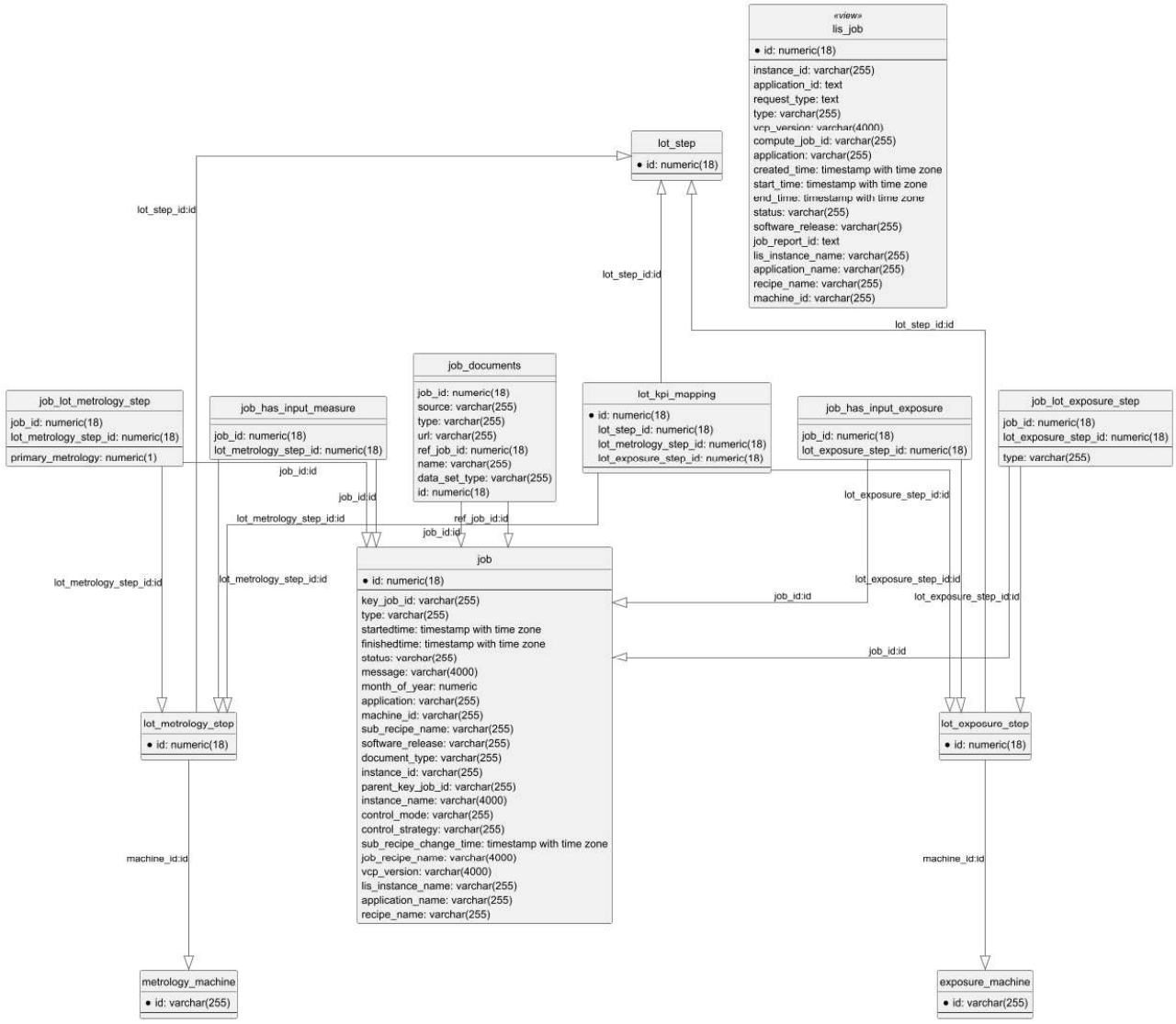


Figure 2.4-6 Core Context Model – Entity Relationship Diagram (Lis Job Tables/Views)

2.4.6 CORE CONTEXT MODEL – RELATION DATABASE MAPPING WITH CANONICAL PATH

Explains how a certain context from Core Context Model Class Diagram (currently being referred using Canonical Path in plugins and Applications) is implemented in Relational Database, how the new interface would look like,how and from where context information can be extracted. In case there are multiple relations between two entities e.g. between Job and Document as inputDocuments and outputDocuments the Canonical Path can't be formed with EntityName only, in such cases relation name is used to form right Canonical Path.

Core Contexts as Canonical Paths	**Core Context as Mapped in Relational Database (PostgreSQL)**
exposureprocessjob_ • reticle_keyreticleid • isreworked • processjobid • lotid • waferlot_keylotid • layerid • processstep_layer_keyphysicallayerid • processstep_productid • equipment_id • starttime • endtime • recipename • softwarerelease • islatestreworked	**EXPOSURE_PROCESS_JOB** (*DB View on **LOT_EXPOSURE_STEP** table*) • key_reticle_id • reworked • process_job_id • lot_id • key_lot_id • layer_id • key_physical_layer_id • product_id • equipment_id • start_time • end_time • recipe_name • software_release • latest_reworked
exposureprocessjob_measures	**LATEST_EXPOSURE** (*DB View on **LOT_KPI_MAPPING** with Aggregation Level 'LOT'*) Same View is depicting all relations between ExposureProcessJob and MeasureProcessJob. **E.g.** If an Exposure is Measured multiple times, the link from every MeasureProcessJob to ExposureProcessJob would be the latestExposure of that individual MeasureProcessJob, while on the other hand the same View is depicting the one to many measures relation from ExposureProcessJob to MeasurereProcessJob.
exposureprocessjob_waferexposureprocessjob_ • waferid • waferprocessingorder • starttime • endtime	**WAFER_EXPOSURE_PROCESS_JOB** (*DB View on **WAFER_EXPOSURE_STEP** table*) • wafer_id • wafer_processing_order • start_time • end_time

• status • wafersequencenr • chuck_id • wafer_keywaferid • isreworked • processjobid • waferprocessjobid	• status • wafer_sequence_number • chuck_id • key_wafer_id • reworked • process_job_id • wafer_process_job_id
measureprocessjob_ • processjobid • lotid • waferlot_keylotid • layerid • processstep_layer_keyphysicallayerid • processstep_productid • equipment_id • starttime • endtime • recipename • softwarerelease • waferrotation • waferlot_waferdiameter	**MEASURE_PROCESS_JOB** (*DB View on **LOT_METROLOGY_STEP** table*) • process_job_id • lot_id • key_lot_id • layer_id • key_physical_layer_id (*in LANA DB, there is no additional context mapping available for Layer context*) • product_id • equipment_id • start_time • end_time • recipe_name • software_release • wafer_load_orientation • wafer_diameter
measureprocessjob_latestexposure	**LATEST_EXPOSURE** (*DB View on **LOT_KPI_MAPPING** with Aggregation Level 'LOT'*)
measureprocessjob_wafermeasureprocessjob_ • waferid • waferprocessingorder • starttime • status • wafer_keywaferid • processjobid • waferprocessjobid	**WAFER_MEASURE_PROCESS_JOB** (*DB View on **WAFER_METROLOGY_STEP** table*) • wafer_id • wafer_processing_order • start_time • status • key_wafer_id • process_job_id • wafer_process_job_id
wafermeasureprocessjob_latestexposure	**WAFER_LATEST_EXPOSURE** (*DB View on **WAFER_KPI_MAPPING***)
wafermeasureprocessjob_measures	**WAFER_LATEST_EXPOSURE** (*DB View on **WAFER_KPI_MAPPING***)

	As explained with 'exposureProcessJob_measures', same applied at Wafer level.
lisjob_ • applicationid • requesttype • id • applicationname • createdtime • starttime • endtime • status • softwarerelease • outputdocuments_keydocumentid (*specific to JobReport as only single JobReport Document per LIS job can exist*) • recipename • machineId	**LIS_JOB** (*DB View on JOB table*) • application_id • request_type • compute_job_id • application_name • created_time • start_time • end_time • status • software_release • job_report_id • recipe_name • machine_id
otasjob_ (*all other common contexts are mapped from same LIS_JOB View with a few exceptions as listed below*) • controlfunctional • controlstrategy	**JOB** Table • bali_control_functions • control_strategy
lisjob_inputdocuments (*for Exposure Document used as input to Lis Job*)	**JOB_HAS_INPUT_EXPOSURE** (*DB View to show if an Exposure is used as Input to a LIS Job*)
lisjob_inputdocuments (*for Measure Document used as input to Lis Job*)	**JOB_HAS_INPUT_MEASURE** (*DB View to show if a Measure is used as Input to a LIS Job*)
lisjob_outputdocuments (*for all output Documents*)	**JOB_DOCUMENTS** *where SOURCE='JOB_OUTPUT_DOCUMENTS'*
lisjob_inputdocuments (*for all input Documents*)	**JOB_DOCUMENTS** *where SOURCE='JOB_INPUT_DOCUMENTS'*
measureprocessjob_inputdocuments_ • keydocumentid • type • subconfiguration • servicedescriptor	**MEASURE_DOCUMENT** (*input=true depicting if the referenced Document* is input to a MeasureProcessJob) referencing **LOT_METROLOGY_STEP** • canonical_key • document_type • sub_configuration • service_descriptor
measureprocessjob_outputdocuments_ • keydocumentid • type • subconfiguration	**MEASURE_DOCUMENT** (*input=false depicting if the referenced Document* is input to a MeasureProcessJob) referencing **LOT_METROLOGY_STEP** • canonical_key

• servicedescriptor	• document_type • sub_configuration • service_descriptor
exposureprocessjob_inputdocuments_ • keydocumentid • type • subconfiguration • servicedescriptor	**EXPOSURE_DOCUMENT** (*input=true depicting if the referenced Document* is input to a ExposureProcessJob) referencing **LOT_EXPOSURE_STEP** • canonical_key • document_type • sub_configuration service_descriptor
measureprocessjob_outputdocuments_ • keydocumentid • type • subconfiguration • servicedescriptor	**EXPOSURE _DOCUMENT** (*input=false depicting if the referenced Document* is input to a ExposureProcessJob) referencing **LOT_EXPOSURE_STEP** • canonical_key • document_type • sub_configuration • service_descriptor
exposureprocessjob_equipment and measureprocessjob_equipment • exposureprocessjob_equipment_id • exposureprocessjob_equipment_namebycustomer • measureprocessjob_equipment_id • measureprocessjob_equipment_namebycustomer	*There is no exact one to one mapping from iACT Context Model and Analytics Database. In Analytics Database there are* **EXPOSURE_MACHINE** *and* **METROLOGY_MACHINE** *with relevant* information. • EXPOSURE_MACHINE.id • EXPOSURE_MACHINE.customername • MEASURE_MACHINE.id • MEASURE_MACHINE.customername
Exceptions: Contexts which are not mapped from Core Context Model to Analytics Database.	
• exposureprocessjob_waferexposureprocessjob_ wafer_waferscribe • measureprocessjob_wafermeasureprocessjob_ wafer_waferscribe • measureprocessjob_processstep_layer_classification • equipment_equipmenttype • equipment_classification	

2.5 FRONT-END SUPPORT

2.5.1 FRONT-END LABEL FOR CORE

Below are the core context columns and corresponding labels that shall be used in the front-end. Labels for domain extensions shall be defined in the corresponding SEG documents.

column (context path)	FE label	valueType
exposureprocessjob_equipment_id	Exposure equipment ID	StringType
exposureprocessjob_id	Lot exposure ID	
exposureprocessjob_reticle_keyreticleid	Exposure Reticle ID	StringType
exposureprocessjob_starttime	Exposure start time	TimeStampType
exposureprocessjob_waferexposureprocessjob_chuck_id	Chuck ID	StringType
exposureprocessjob_waferexposureprocessjob_exposurelogicalwafer_waferexposureresult	Wafer Status	StringType
exposureprocessjob_waferexposureprocessjob_measurelogicalwafer_strategyid	Strategy ID	StringType
exposureprocessjob_waferexposureprocessjob_waferid	Exposure wafer ID	StringType
exposureprocessjob_waferexposureprocessjob_wafersequencenr	Exposure Order	IntegerType
exposureprocessjob_waferlot_keylotid	Exposure lot ID	StringType
measureprocessjob_equipment_id	Measurement equipment ID	StringType
Measureprocessjob_id	Lot measure ID	
measureprocessjob_layerid	Measurement layer ID	StringType
measureprocessjob_processstep_productid	Product ID	StringType
measureprocessjob_starttime	Measurement start time	TimeStampType
measureprocessjob_waferlot_keylotid	Measurement lot ID	StringType
measureprocessjob_wafermeasureprocessjob_waferid	Measurement wafer ID	StringType

2.5.2 Error! Not a valid link. FRONT-END LABEL GUIDELINES FOR DOMAIN EXTENSIONS

This is a placeholder to document the guidelines that need to be followed by domain plugins.

1

3 REQUIREMENTS

3.1 SOFTWARE INTERFACE REQUIREMENTS

3.2 FUNCTIONAL REQUIREMENTS

The following requirements from the leading SD, D000987766/06, are applicable for the iACT context model

3.2.1.1 CONTEXT

ID	Status	Design decision	SF
CO_01	Accepted	**Domain model** A. The platform shall have a domain model with domain entities and mutual associations. Imported or processed performance data shall be given context by means of instantiations of these entities. Rationale: To turn raw data elements into real usable information. B. The domain model shall comply to the common meta model described in D000987766 *Rationale*: Because the platform is agnostic about the model extensions that are delivered by the plugins, it cannot have specific behavioral rules based on that. Hence, some behavioral rules shall be based on meta model level aspects instead. See D000987766 about coarse data selection, and examples of such behavior rules.	205
CO_02	Accepted	**Core domain model** A. The platform shall contain the definition of a core wafer fab process domain model, i.e. a model of core entities as depicted in D000987766. *Rationale*: These core entities have explicit meaning within the platform's functionality and corresponding implementation	205
CO_03	Accepted	**Domain model extension** A. The domain model as used by the platform shall be extendable. The core model shall serve as a means to have application domains hook their specific entities, attributes or relations into. Extensions shall be possible by a plugin construction. *Rationale*: Definition of a complete ASML wide domain model is considered unrealistic. Instead a decoupled approach is preferred where each application domain can add their model extensions to associate their data to the fab process model. B. Each application/performance domain has its own namespace for its model extensions, that is fixed (hardcoded) in the platform. Rationale: To avoid ambiguity by possibly conflicting extensions. Hardcoded because these domains are rather stable. C. For process jobs or process job to wafer combinations it shall be possible to extend the domain model with new attributes. i.e., context that is provided by the customer system, typically in terms of key-value name pairs that have no further semantic meaning for the platform implementation itself. *Rationale*: This abstract context is still valuable for data selection/filtering by an end-user that has knowledge about its semantical meaning. The content is customer specific and deploying changes must not be affected by any platform development lead time.	205

ID	Status	Design decision	SF
CO_04	Accepted	**Context types** A. The application domains shall be enabled to register context types as defined in D000987766 to become available in the UI for data selection. The platform itself shall handle the "registration" of the context related to core domain entities. *Rationale*: The complete domain model with all its entity relations represents an unpractically large amount possible context types (read paths to the entity attributes). Therefore the important/relevant context types are to be specified explicitly. The context type definitions include a label to be used in the UI to make it more user friendly. (Also note the relabeling option, see UF_02.) D. The governance for these context types in terms of naming conventions, consistencies, non-duplicates, shall be managed via model definitions in the ASML Data Governance Center (Collibra tool). C. The platform shall provide a UI construct to define additional context by means of tagging (naming) an active fine data selection (i.e., the context filters and in-plot selection). The tags shall be applied to either 1a) the selected KPI values or 1b) the selected context values. User defined context shall be available for 2a) the active workspace only, 2b) a user, i.e. shared between the user's workspaces or 2c) shared with "all" users. It shall be handled as any context, and thus made available in 3a) the inventory and 3b) the data warehouse for slicing and dicing. Private versus public scope shall be handled via the application assets management. Private context handled in similar way as for private datasets resulting from processing flows, or privately imported data (manually uploaded).Public context in a similar way as for "golden" presets/recipes, managed by super-users. *Rationale*: This construct enables a powerful generic mechanism for the user to do use case specific slice and dicing without the need of building in specialties. E.g. a (radial) wafer region/zone context can be created in this way by means of doing in-plot selection. (Zone definition as product related assets (see asset management in D000987766) is out of scope.) Example: Assume user tags three wafer level values for two different machines. 1a) means that in that case these three wafer values (KPIs) get an additional context in terms of the given tag. 1b) means that for this combination of selected wafer IDs and machine IDs there will be a context value of given tag name that represents this context combination.	205, 208
	Draft	B. Platform shall validate there are no different context types (paths) that point to the same attribute. Context types shall be guaranteed to be canonical. *Rationale*: Enable the framework to recognize context types from different datasets to be equal. Otherwise this would hamper dataset virtualization, i.e. optimization of dataset selection, unionizing and/or joining. <See OI-31.>	
CO_05	Accepted	**Validity and health** A. Data *record* health (as determined by data preparation plugins, see DI_04) and KPI value validity, like other contextual or meta information about a KPI value, shall be handled via context attributes. *Rationale*: Having a data model with only two "types" of data (KPI data and context data) simplifies the design for a generic implementation that needs to be largely agnostic for the specific domain's specialties. B. Boolean validity (i.e., valid versus invalid, hence not the invalidity reason) and record health shall be elevated concepts from visualization point of view, i.e. having dedicated UI constructs (e.g. a toggle to show/hide invalid measurements) *Rationale*: To give these aspect sufficient emphasis in comparison to "regular" contexts. To make the user aware of unhealthy data and thus avoiding wrong conclusions based on that.	205 208
CO_06	Accepted	**Context storage** A. Context at wafer level or above shall be stored in the Inventory. This includes both 1) public as well as 2) private context resulted from executed processing flows and tagging construct mentioned under CO_04.C. B. Context at sub-wafer level (like field level context such as scan direction) shall be stored in the data warehouse only. Hence not in the Inventory. *Rationale:* To not "overload" the inventory database.	205

ID	Status	Design decision	SF
		C. The data warehouse shall also duplicate (a subset of) context from the inventory. This shall be the context info to be used by visualization and processing functions. Rationale: Performance. To avoid joins between datasets and context from the inventory. <OI-8: For MS1+2 only context from data warehouse is used. For > MS2 a join with Inventory context could be feasible as well.>	

3.3 NON-FUNCTIONAL REQUIREMENTS

ID	Status	Design decision	SF
PL_01	Accepted	**Semantic Versioning** This EPS explicitly specify the context model version. Major (=breaking) changes require an update + review of this EPS with all relevant stakeholders	205

3.4 DESIGN CONSTRAINTS

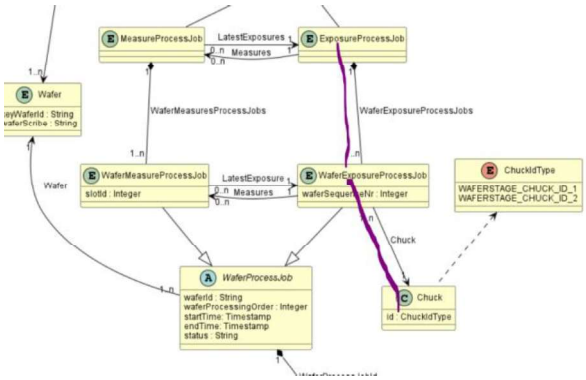
4 APPENDICES

APPENDIX A USAGE OF CONTEXT MODEL IN THE DATASET REFERENCE SPECIFICATION

Two types of columns can be specified in a dataset reference file:

- Context columns: The name of a context column represent a path in the Context Model. The anchor point for a context path can be any job type. See the examples below where context column names start at measureprocessjob or exposureprocessjob as source, then the path is build using the arc(association name), and ends with the attribute name of the target concept.

Context column names	Context Model **path example**

{
"id": "d94af30a-cdc7-4a0c-b321-737a49ee1a27",
"name": "measureprocessjob_waferlot_keylotid",
"master": true,
"type": {
"name": "StringType"
},
"columnType": "context"
},
{
"id": "8883ca1f-a455-4db5-993b-dfcdbcc059c9",
"name": "measureprocessjob_starttime",
"master": true,
"type": {
"name": "TimestampType"
},
"columnType": "context"
},
{
"id": "1dec8211-cfa3-48a6-83aa-47f7bf29219a",
"name": "exposureprocessjob_waferexposureprocessjob_chuck_id",
"master": true,
"type": {
"name": "StringType"
},
"columnType": "context"
},
	
	measureprocessjob_waferexposureprocessjob_chuck_id

Note: This convention may change with the implementation of the extension mechanism for Context Model/WDaM.(See OI08)

- KPI Columns: KPI column names are not fixed to paths in the Context Model. The association of a KPI column with context columns shall be done by setting the values for GroupByContext and MaxGranularity in terms of context columns, see the picture below. Beside GroupByContext and MaxGranularity, KPI specification is out of the scope of Context Model.


{
"columnType": "kpi",
"id": "e175648c-945b-4bcf-9e2a-8cf9b1e9cd36",
"name": "overlay_y",
"master": false,
"type": {
"name": "DoubleType"
},
"kpi": {
"id": "e175648c-945b-4bcf-9e2a-8cf9b1e9cd36",
"groupByContexts": [],
"kpiType": {
"id": "1797ab36-c3fe-11ec-9d64-0242ac120002",
"name": "Overlay",
"performanceDomain": "OVERLAY",
"quantity": {
"id": "27758d16-c3fe-11ec-9d64-0242ac120002",
"domain": "OVERLAY",
"maxGranularity": {
"X": [
"measureprocessjob_lotid", "measureprocessjob_starttime",
"measureprocessjob_equipment_equipmentid", "measureprocessjob_wafermeasureprocessjob_waferid",
"measureprocessjob_wafermeasureprocessjob_measurement_intrafieldposition_field_center_x"
],
"Y": [
"measureprocessjob_lotid", "measureprocessjob_starttime",
"measureprocessjob_equipment_equipmentid", "measureprocessjob_wafermeasureprocessjob_waferid",
"measureprocessjob_wafermeasureprocessjob_measurement_intrafieldposition_field_center_y"
]
},
"quantityType": "OVERLAY",
"units": [
"NM"
],

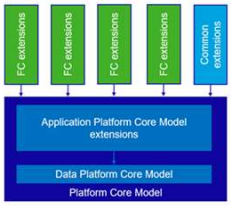
For more examples on using Context Model for dataset reference files see: [foundation-data-reference-plugin - Bitbucket PRD EID (asml.com)]

APPENDIX B COMMONALITY IN CONTEXT EXTENSIONS

Context entities are used in multiple use cases, see the picture below that illustrates how current domain plugins are using the entities in the context model.

To be mapped to FCs

Process	Entity	Overlay	CD	Focus	Overlay Margin	WA Alignment	PnP
Exposure	Chuck	x			x	x	
	Die				x		
	Field	x			x	x	x
	Reticle	x					
	Image	x			x		
	Mark					x	
Measure	Target	x					x
	Cutline				x		



Hence, there is the need to manage commonality and these are the alternatives:

1. Move common context to a common context package that can be used by all other (SF) packages
   - Pros: Locality
   - Cons: Unclear ownership and large change impact
2. Common context is owned/included in one SF package and shared with the other SF packages
   - Pros: Explicit ownership, less change impact.
   - Cons: Horizontal dependencies between SFs
3. Multiple SF packages specify the same context and we have an elevated association in the common package that associates the same contexts
   - Pro: Loose coupling
   - Cons: Implicit redundancy and additional business logic

The current approach goes in the direction of alternative 1, i.e. the sub-wafer context model contains shared entities. This needs to be revised with future iterations.